



**INSTITUTO SUPERIOR
TECNOLÓGICO QUITO**

Lenguaje de programación 3



Ing. Eduardo Avilés



eduardo.aviles@itq.edu.ec





Controllers - tienda.js

JS tienda.js

backend > controllers > JS tienda.js > ...

```
1  'use strict'
2  var Producto = require("../models/producto");
3  var path=require('path');
4  var fs=require('fs');
5
6  var controller={
7    home:function(req,res){
8      return res.status(200).send(
9        "<h1>hola mundo</h1>"
10     );
11   },
12   saveProducto:function(req,res){
13     var producto=new Producto();
14     var params=req.body;
15
16     producto.marca=params.marca;
17     producto.modelo=params.modelo;
18     producto.ram=params.ram;
19     producto.rom=params.rom;
20     producto.precio=params.precio;
21     producto.imagen=null;
22     // save() también devuelve una promesa
23     producto.save()
24       .then(productoStored => {
25         if(!productoStored) return res.status(404).send({message:'No se ha guardado el producto'});
26         return res.status(200).send({producto:productoStored});
27       })
28       .catch(err => {
29         return res.status(500).send({message:'Error al guardar', error: err});
30       });
31   },
```

EXPLORER

EXAMPLE

backend

controllers

JS tienda.js

models

node_modules

routes

uploads

JS app.js

JS index.js

package-lock.json

package.json





Controllers - tienda.js

```
32   getProductos:function(req,res){
33     Producto.find({}).sort().exec()
34     .then(productos => {
35       if(!productos || productos.length === 0)
36         return res.status(404).send({message:'No hay productos para mostrar'});
37       return res.status(200).send({productos});
38     })
39     .catch(err => {
40       return res.status(500).send({message:'Error al recuperar los datos', error: err});
41     });
42   },
43   getProducto:function(req,res){
44     var productoId=req.params.id;
45
46     Producto.findById(productoId) // findById también devuelve una promesa
47     .then(producto => {
48       if(!producto) return res.status(404).send({message:'El producto no existe'});
49       return res.status(200).send({producto});
50     })
51     .catch(err => {
52       // Mongoose puede lanzar un CastError si el ID no es válido
53       if (err.name === 'CastError') {
54         return res.status(404).send({message:'Formato de ID de producto no válido.'});
55       }
56       return res.status(500).send({message:'Error al recuperar los datos', error: err});
57     });
58   },
```



Controllers - tienda.js

```
59 deleteProducto:function(req,res){
60     var productoId=req.params.id;
61     // Mongoose recomienda findByIdAndDelete en lugar de findByIdAndRemove
62     Producto.findByIdAndDelete(productoId)
63         .then(productoRemoved => {
64             if(!productoRemoved)
65                 return res.status(404).send(
66                     {message:'No se puede eliminar el producto porque no existe'});
67             return res.status(200).send(
68                 {producto:productoRemoved, message: 'Producto eliminado correctamente'});
69         })
70         .catch(err => {
71             if (err.name === 'CastError') {
72                 return res.status(404).send(
73                     {message:'Formato de ID de producto no válido para eliminar.'});
74             }
75             return res.status(500).send({message:'Error al eliminar los datos', error: err});
76         });
77 },
78 updateProducto:function(req,res){
79     var productoId=req.params.id;
80     var update=req.body;
81     Producto.findByIdAndUpdate(productoId,update,{new:true})
82         .then(productoUpdate => {
83             if(!productoUpdate) return res.status(404).send(
84                 {message:'El producto no existe para actualizar'});
85             return res.status(200).send({producto:productoUpdate});
86         })
87     }
```



Controllers - tienda.js

```
87         .catch(err => {
88             if (err.name === 'CastError') {
89                 return res.status(404).send(
90                     {message: 'Formato de ID de producto no válido para actualizar.'});
91             }
92             return res.status(500).send({message: 'Error al actualizar los datos', error: err});
93         });
94     },
95     uploadImagen: function (req, res) {
96         var productoId = req.params.id;
97         var fileName = 'Imagen no subida...';
98
99         if (req.files) {
100             var filePath = req.files.imagen.path;
101             var file_split = filePath.split('\\');
102             var fileName = file_split[file_split.length - 1];
103
104             var extSplit = fileName.split('\\.');
105             var fileExt = extSplit[extSplit.length - 1];
106
107             if (fileExt === 'png' || fileExt === 'jpg' || fileExt === 'jpeg' || fileExt === 'gif') {
108                 Producto.findByIdAndUpdate(productoId, {imagen: fileName}, {new: true})
109                     .then(productoUpdated => {
110                         if (!productoUpdated) {
111                             // Eliminar archivo temporal si no se encuentra el producto
112                             fs.unlink(filePath, (unlinkErr) => {
113                                 return res.status(404).send(
114                                     {message: 'El producto no existe y no se subió la imagen'});
115                             });
116                         }
117                     });
118             }
119         }
120     }
121 }
```



Controllers - tienda.js

```
115         } else {
116             return res.status(200).send({producto: productoUpdated});
117         }
118     })
119     .catch(err => {
120         // Eliminar archivo temporal si hay error de DB
121         fs.unlink(filePath, (unlinkErr) => {
122             if (unlinkErr) console.error('Error al eliminar archivo temporal:', unlinkErr);
123             if (err.name === 'CastError') {
124                 return res.status(404).send(
125                     {message: 'Formato de ID de producto no válido para subir imagen.'});
126             }
127             return res.status(500).send({
128                 message: 'Error al subir la imagen o actualizar el producto', error: err});
129             });
130         });
131     } else {
132         fs.unlink(filePath, (err) => {
133             if (err) console.error('Error al eliminar archivo con extensión no válida:', err);
134             return res.status(200).send(
135                 {message: 'La extensión no es válida, archivo eliminado.'});
136         });
137     }
138 } else {
139     return res.status(400).send({
140         message: 'No se ha subido ninguna imagen.'
141     });
142 }
143 },
```




Controllers - tienda.js

```
144     getImagen:function(req,res){
145         var file=req.params.imagen;
146         var path_file='./uploads/'+file;
147         // si fs.exists está obsoleto, mejor usar fs.promises.access o fs.stat dependiendo si da error o no
148         fs.exists(path_file,(exists)=>{
149             if(exists){
150                 return res.sendFile(path.resolve(path_file));
151             }else{
152                 return res.status(404).send({message:'No existe la imagen'});
153             }
154         });
155     }
156 }
157 module.exports=controller;
```





```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  node - backend + v [icon] [icon] ... ^ X

PS C:\Users\USUARIO\Documents\Instituto Tecnológico Superior Quito\Lenguaje de programación 3\Parcial 3\Example\backend> npm start

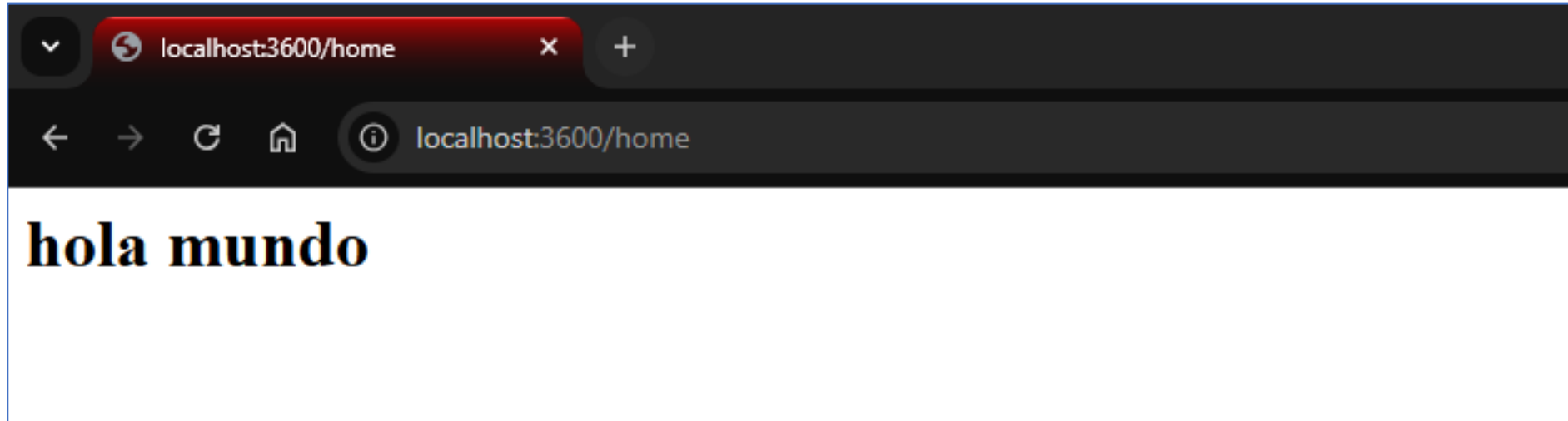
> backend@1.0.0 start
> nodemon index.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Conexion establecida con la bdd
Conexion establecida en el url: localhost:3600
```



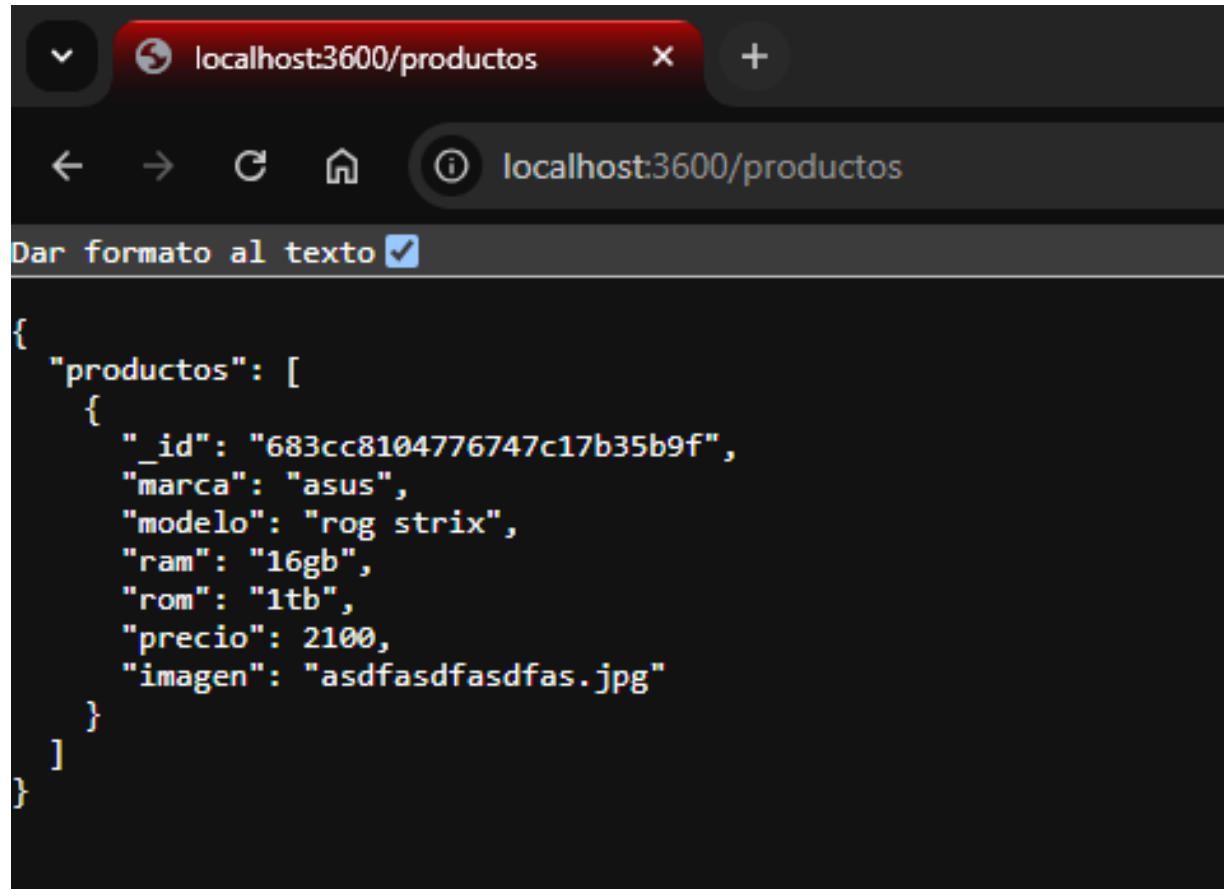


Resultado



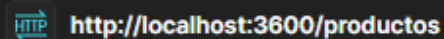


Resultado



```
{
  "productos": [
    {
      "_id": "683cc8104776747c17b35b9f",
      "marca": "asus",
      "modelo": "rog strix",
      "ram": "16gb",
      "rom": "1tb",
      "precio": 2100,
      "imagen": "asdfasdfasdfas.jpg"
    }
  ]
}
```





GET <http://localhost:3600/productos>

Params

Authorization

Headers (6)

Body

Scripts

Tests

Settings

Cookies

Query Params

Body

Cookies

Headers (11)

Test Results

200 OK

25

22 ms • 647 B • |

{ } JSON ▾

▶ Preview

Visualize

```

1  {
2      "productos": [
3          {
4              "_id": "683cc8104776747c17b35b9f",
5              "marca": "asus",
6              "modelo": "rog strix",
7              "ram": "16gb",
8              "rom": "1tb",
9              "precio": 2100,
10             "imagen": "asdfasdfasdfas.jpg"
11         }
12     ]
13 }

```



Taller 01

Consultar y exponer sobre códigos de estado HTTP





Ing. Eduardo Avilés



eduardo.aviles@itq.edu.ec



0958919713